

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-2

Proyecto 2

Juego en Assembly sobre Pochoco SoC

Grupo: 6
Integrantes:

Manuel Caroca
Julian Rodriguez
Alfonso Villanueva

Profesor de Cátedra: Jorge Gómez

21 de octubre de 2026

Índice

1. Introducción y objetivos **2**

2. Modificaciones al hardware y mapa de memoria **2**

2.1. Acceso a periféricos mediante direcciones de memoria 2

2.2. Mapa de memoria utilizado por el juego 2

2.3. Contador libre de ciclos 3

3. Diseño y funcionamiento del juego en Assembly **4**

3.1. Organización general y registros 4

3.2. Inicialización y comienzo de la partida 5

3.3. Encendido de los cuatro LEDs y espera 6

3.4. Selección del LED objetivo 6

3.5. Medición y respuesta correcta 7

3.6. Conversión a BCD y actualización del resultado 8

3.7. Respuesta incorrecta 9

3.8. Décima respuesta y promedio 10

3.9. Reinicio de una nueva partida 10

3.10. Rebotes y pulsaciones anticipadas 11

4. Temporización y cálculos **11**

4.1. Frecuencia y constantes 11

4.2. Resta modular y desbordamiento 11

4.3. Selección pseudoaleatoria del LED 12

5. Diseño e implementación del assembler **13**

5.1. Flujo de compilación y programación 13

6. Validación y resultados en FPGA **13**

6.1. Limitaciones observables 16

7. Conclusiones **16**

1. Introducción y objetivos

El objetivo del proyecto es implementar un juego de reacción que se ejecute en el procesador Espino Core del Pochoco SoC y utilice los periféricos de la NANDLAND GoBoard. En cada intento se encienden los cuatro LEDs durante al menos tres segundos; después se enciende un LED objetivo y el jugador debe presionar el switch correspondiente. El programa mide el tiempo de respuesta, indica una respuesta incorrecta y repite ese intento sin borrar los aciertos anteriores. Después de diez respuestas correctas muestra el promedio.

La solución se divide entre hardware descrito en Verilog y software escrito en Assembly RV32E. El hardware proporciona los registros de entrada y salida mapeados en memoria (MMIO) y un contador libre de ciclos. El programa organiza las rondas, consulta los switches, calcula los tiempos y prepara los valores para los displays.

Este informe presenta la arquitectura del SoC, el mapa de memoria utilizado, las modificaciones al hardware, el funcionamiento del programa, el cálculo de los tiempos, el assembler y el estado de la validación disponible. Los requisitos funcionales se basan en el enunciado del proyecto.

2. Modificaciones al hardware y mapa de memoria

2.1. Acceso a periféricos mediante direcciones de memoria

El juego accede a los periféricos mediante lecturas y escrituras en direcciones específicas (memory-mapped input/output). El procesador utiliza instrucciones de acceso a memoria, pero el destino puede ser un periférico en vez de una posición de RAM.

La inicialización del programa carga la dirección base de los periféricos en `x2`:

```
1    lui  x2, 0x80000
```

Listado 1: Carga de la dirección base de los periféricos.

Una expresión como `4(x2)` calcula la dirección `0x80000000 + 4 = 0x80000004`. El desplazamiento no modifica el valor de `x2`.

2.2. Mapa de memoria utilizado por el juego

La Tabla 1 resume las direcciones utilizadas por `sw/game_1.s` y las operaciones que realiza el programa en cada una.

Tabla 1: Direcciones MMIO utilizadas por el juego.

Dirección	Expresión	Acceso	Ancho útil	Función
`0x80000000`	`0(x2)`	Escritura	8 bits	Display: dos grupos de cuatro bits con las cifras a mostrar.
`0x80000004`	`4(x2)`	Escritura	4 bits	LEDs: los bits 0 a 3 forman la máscara de encendido.
`0x80000008`	`8(x2)`	Lectura	4 bits	Switches: los bits 0 a 3 indican las entradas activas en alto.
`0x8000000C`	`12(x2)`	Lectura	32 bits	Contador libre de ciclos.

Los desplazamientos de las instrucciones están escritos en decimal. Por eso, `12(x2)` corresponde al desplazamiento hexadecimal `0x0C`. Las instrucciones `lw` y `sw` transfieren palabras de 32 bits, aunque el periférico almacena solo los bits que necesita: 8 para los dos dígitos, 4 para los LEDs y 32 para el contador.

El decodificador de `pochoco_periph` utiliza `addr_i[7:2]`, por lo que los dos bits inferiores no seleccionan otro registro. El programa utiliza las direcciones alineadas que aparecen en la Tabla 1.

2.2.1. LEDs y switches

Para encender los cuatro LEDs se escribe el valor decimal 15, cuya representación en los cuatro bits utilizados es 1111_2. Para seleccionar uno solo se escribe 1, 2, 4 u 8. Estos valores activan, respectivamente, las máscaras 0001_2, 0010_2, 0100_2 y 1000_2. Escribir cero apaga los LEDs.

El programa considera que una lectura igual a cero significa que ningún switch está presionado. Durante una respuesta compara la palabra leída con la máscara completa del LED objetivo. No aplica una máscara adicional a `x4` antes de comparar; por lo tanto, presionar dos switches a la vez produce un valor distinto de cualquiera de las cuatro máscaras de un solo bit y se considera un error.

2.2.2. Displays de siete segmentos

El programa escribe dos dígitos empaquetados en los ocho bits inferiores del registro del display. Los cuatro bits superiores se conectan al primer dígito y los cuatro inferiores al segundo.

Cada grupo de cuatrobits contiene una cifra decimal entre 0 y 9. Por ejemplo, el valor `0x42` solicita mostrar las cifras 4 y 2. Para señalar un error se escribe `0xEE`, con lo que se solicita mostrar `EE`. Al iniciar una partida se escribe cero y se muestra `00`.

2.3. Contador libre de ciclos

El juego consulta un contador de 32 bits mediante `lw` en `0x8000000C`. El contador se incrementa en cada flanco ascendente del reloj.

Una lectura obtiene una muestra del contador y la guarda en un registro del procesador. El contador continúa avanzando; la muestra no lo detiene. Por ejemplo, `x5` puede conservar el comienzo de una espera mientras `x9` recibe lecturas posteriores. La diferencia entre ambas muestras permite medir el intervalo.

El juego no escribe en el contador para reiniciarlo en cada ronda. Utiliza restas entre muestras, por lo que no necesita que el contador comience en cero al iniciar una partida.

3. Diseño y funcionamiento del juego en Assembly

3.1. Organización general y registros

La secuencia general es: esperar el inicio, preparar una ronda, seleccionar un LED, detectar la respuesta y decidir entre repetir el intento o acumular un acierto. Al completar diez respuestas correctas, espera dos segundos, calcula el promedio y lo deja visible hasta que se inicie otra partida.

Las etiquetas identifican posiciones del programa; no son variables ni instrucciones ejecutables. Los saltos de la forma `jal x0, etiqueta` cambian el flujo de ejecución sin guardar una dirección de retorno, porque el destino del enlace es `x0`.

Tabla 2: Uso de los registros del juego.

Registro	Función en el programa
`x0`	Valor constante cero; también descarta el enlace de los saltos.
`x1`	Muestra del contador usada como semilla de la partida.
`x2`	Dirección base de los periféricos: `0x80000000`.
`x3`	Máscara del LED objetivo; se reutiliza para enviar `0xEE`.
`x4`	Lectura de switches y comparaciones auxiliares.
`x5`	Muestra inicial de una espera o valor empaquetado para el display.
`x6`	Umbral de ciclos para una espera o conversión.
`x7`	Cantidad de respuestas correctas de la partida.
`x8`	Muestra del contador al comenzar la medición de una respuesta.
`x9`	Muestra reciente del contador; luego diferencia o resto de ciclos.
`x10`	Diferencia temporal, tiempo en décimas o promedio.
`x11`	Constantes auxiliares, principalmente 10 o 99.
`x12`	Suma de los tiempos correctos, en décimas.
`x13`	No se utiliza en esta versión.
`x14`, `x15`	Cociente y resto; luego decenas y unidades BCD.

La función de un registro puede cambiar entre bloques. Por ejemplo, `x10` almacena una diferencia de ciclos durante una espera y un tiempo en décimas después de medir una respuesta. La suma que se necesita para el promedio permanece en `x12`.

3.2. Inicialización y comienzo de la partida

```
_start:
    lui  x2, 0x80000
    addi x7, x0, 0
    addi x12, x0, 0
    sw   x0, 0(x2)
    sw   x0, 4(x2)

wait_start_press:
    lw   x4, 8(x2)
    beq  x4, x0, wait_start_press
    lw   x1, 12(x2)

wait_start_release:
    lw   x4, 8(x2)
    bne  x4, x0, wait_start_release
    jal  x0, round_start
```

Listado 2: Inicialización y detección del inicio.

La instrucción `lui` carga en `x2` la base MMIO. Las dos instrucciones `addi` ponen en cero el número de aciertos `x7` y la suma de tiempos `x12`. Después, dos `sw` muestran `00` y apagan los LEDs.

En `wait_start_press`, `lw` lee los switches. `beq` compara `x4` con cero y mantiene al programa en el bucle mientras no se presione ninguno. Cuando se detecta una pulsación, se lee el contador y se guarda una muestra en `x1`; esa muestra se conserva como semilla durante la partida.

El segundo bucle espera a que todos los switches vuelvan a cero. De esta manera, la pulsación de inicio no se cuenta como respuesta. Una vez liberados los switches, `jal x0, round_start` transfiere la ejecución a la primera ronda sin almacenar una dirección de retorno.

3.3. Encendido de los cuatro LEDs y espera

Cada intento comienza encendiendo los cuatro LEDs. Luego se guarda una muestra del contador y se espera hasta que hayan pasado al menos tres segundos:

```
round_start:
    addi x3, x0, 15
    sw   x3, 4(x2)
    lw   x5, 12(x2)
    lui  x6, 0x4787
    addi x6, x6, -1856

wait_three_seconds:
    lw   x9, 12(x2)
    sub  x10, x9, x5
    bltu x10, x6, wait_three_seconds
```

Listado 3: Encendido de los cuatro LEDs y espera basada en el contador.

`addi x3, x0, 15` prepara la máscara 1111_2, y `sw` la escribe en el registro de LEDs. `lw x5, 12(x2)` captura el inicio de la espera. El umbral de ciclos queda en `x6`. En cada vuelta, `lw` obtiene una muestra reciente en `x9`; `sub` calcula la diferencia y `bltu` vuelve a esperar mientras esa diferencia sea menor que el umbral.

La espera no termina necesariamente en el ciclo exacto del umbral, porque el programa verifica la condición al volver a leer el contador. Por eso los LEDs permanecen encendidos durante al menos tres segundos desde la muestra inicial, más el margen del sondeo y las instrucciones ejecutadas. La escritura que los enciende ocurre antes de esa muestra.

3.4. Selección del LED objetivo

Al terminar la espera, el programa calcula un índice entre 0 y 3 a partir de la semilla de inicio, la muestra reciente del contador y el número de aciertos:

```
    add  x3, x9, x1
    add  x3, x3, x7
    andi x3, x3, 3

    beq  x3, x0, choose_led0
    addi x4, x0, 1
    beq  x3, x4, choose_led1
    addi x4, x0, 2
```

```
    beq  x3, x4, choose_led2
    addi x3, x0, 8
    jal  x0, target_ready

choose_led0:
    addi x3, x0, 1
    jal  x0, target_ready

choose_led1:
    addi x3, x0, 2
    jal  x0, target_ready

choose_led2:
    addi x3, x0, 4
```

Listado 4: Cálculo del índice y conversión a máscara de LED.

Las dos instrucciones `add` suman la muestra de contador `x9`, la semilla `x1` y el número de aciertos `x7`. `andi x3, x3, 3` conserva los dos bits inferiores y produce un índice entre 0 y 3. Las ramas `beq` seleccionan una de cuatro máscaras: 1, 2, 4 u 8. Cada máscara tiene un único bit activo, por lo que enciende un solo LED.

3.5. Medición y respuesta correcta

Una vez calculada la máscara, el programa guarda el inicio lógico de la medición, enciende el LED objetivo y consulta los switches:

```
target_ready:
    lw   x8, 12(x2)
    sw   x3, 4(x2)

wait_for_answer:
    lw   x4, 8(x2)
    beq  x4, x0, wait_for_answer
    lw   x9, 12(x2)
    bne  x4, x3, wrong_button
```

Listado 5: Activación del LED objetivo y lectura de la respuesta.

`x8` recibe la muestra de inicio antes de escribir la máscara objetivo en los LEDs. El bucle de respuesta vuelve a leer los switches hasta obtener un valor distinto de cero. A continuación, el programa captura

otra muestra en `x9`. `bne` compara la lectura completa con la máscara objetivo: si son distintas, la respuesta es incorrecta; si son iguales, continúa por el camino de respuesta correcta.

La medición incluye el intervalo entre las dos muestras del contador. Por tanto, también incluye la latencia de las instrucciones y del sondeo de switches; no mide el instante físico exacto de encendido o contacto.

Para una respuesta correcta, primero se apagan los LEDs y se resta la muestra inicial a la final. Después, el código convierte ciclos a décimas de segundo:

```
    sw   x0, 4(x2)
    sub  x9, x9, x8
    addi x10, x0, 0
    lui  x6, 0x262
    addi x6, x6, 1440

cycles_to_tenths:
    bltu x9, x6, tenths_ready
    sub  x9, x9, x6
    addi x10, x10, 1
    addi x11, x0, 99
    bne  x10, x11, cycles_to_tenths
```

Listado 6: Conversión de ciclos a décimas con límite de 99.

Cada resta de 2 500 000 ciclos representa una décima completa. El registro `x10` cuenta cuántas restas se hicieron. El ciclo termina cuando quedan menos ciclos que una décima o cuando `x10` llega a 99. Los ciclos restantes se descartan, por lo que se trunca y no se redondea:

El límite permite representar como máximo 99 décimas, es decir, 9,9 s. No limita cuánto puede esperar el programa por una respuesta; solo limita el valor que muestra y acumula después de detectarla.

3.6. Conversión a BCD y actualización del resultado

El tiempo en `x10` es un número decimal de décimas. Para mostrar sus decenas y unidades, el programa resta diez repetidamente:

```
tenths_ready:
    addi x14, x0, 0
    add  x15, x0, x10
    addi x11, x0, 10

response_to_bcd:
```

```
7     bltu x15, x11, response_bcd_ready
8     sub  x15, x15, x11
9     addi x14, x14, 1
10    jal  x0, response_to_bcd
11
12 response_bcd_ready:
13    add  x5, x14, x14
14    add  x5, x5, x5
15    add  x5, x5, x5
16    add  x5, x5, x5
17    add  x5, x5, x15
18    sw   x5, 0(x2)
```

Listado 7: Separación de decenas y unidades.

`x14` comienza en cero y cuenta las decenas. `x15` comienza como una copia de `x10` y, al restarle diez en cada iteración, termina con las unidades restantes.

Las cuatro instrucciones `add` que duplican el valor multiplican `x14` por 16. Al agregar las unidades se forma $p = 16d + u$, es decir, las decenas en los cuatro bits superiores y las unidades en los cuatro bits inferiores. El resultado se escribe en el registro del display.

Por ejemplo, 42 décimas se muestran como `42`. El valor numérico 42 es `0x2A`; su empaquetado BCD es `0x42`. Por eso el programa acumula `x10`, no el valor empaquetado `x5`.

Después de presentar el resultado, el programa suma `x10` a `x12` e incrementa `x7`. La suma almacena el tiempo de la respuesta en décimas, y el contador indica cuántas respuestas correctas se han completado. Luego el programa espera a que se liberen todos los switches. Si todavía faltan respuestas correctas, vuelve a `round_start`; si `x7` llegó a diez, continúa hacia la pausa y el promedio.

3.7. Respuesta incorrecta

Cuando la lectura de switches no coincide con la máscara del LED, el programa ejecuta el siguiente camino:

```
1  wrong_button:
2     addi x3, x0, 238
3     sw   x3, 0(x2)
4     sw   x0, 4(x2)
5
6  wait_wrong_release:
7     lw   x4, 8(x2)
8     bne  x4, x0, wait_wrong_release
```

```
9       jal  x0, round_start
```

Listado 8: Indicador de error y repetición del intento.

El valor decimal 238 es `0xEE`; al escribirlo en el display, se solicita mostrar `EE`. Los LEDs se apagan y el programa espera que se liberen los switches antes de comenzar nuevamente la ronda. En el camino de error no se incrementa `x7` ni se suma un tiempo a `x12`, por lo que se conservan los aciertos anteriores y se repite el intento actual.

El valor `EE` permanece en el display hasta que otra parte del programa escriba un nuevo valor. No hay una pausa exclusiva para mostrar el error; su duración depende del tiempo hasta la siguiente escritura.

3.8. Décima respuesta y promedio

Después de la décima respuesta correcta y de liberar el switch, el programa espera dos segundos antes de calcular el promedio. Durante ese intervalo no escribe en el display, por lo que se mantiene visible el resultado de la décima respuesta.

```
1  pause_before_average:
2      lw   x5, 12(x2)
3      lui  x6, 0x2faf
4      addi x6, x6, 128
5
6  wait_average_pause:
7      lw   x9, 12(x2)
8      sub  x10, x9, x5
9      bltu x10, x6, wait_average_pause
```

Listado 9: Pausa antes de calcular el promedio.

La suma de los diez resultados permanece en `x12`. El programa divide esa suma por diez usando restas sucesivas; el cociente es el promedio entero en décimas.

El mismo procedimiento de separación y empaquetado BCD se usa para mostrar el promedio. El promedio se trunca a una décima, porque se divide con aritmética entera. También se trunca cada respuesta individual al convertir los ciclos a décimas.

Por ejemplo, para diez resultados cuya suma sea 145 décimas, el promedio entero es 14 décimas. El display muestra `14`, equivalente a 1,4 s. El valor exacto antes del truncamiento sería 1,45 s.

3.9. Reinicio de una nueva partida

Después de mostrar el promedio, el programa espera una nueva pulsación. Toma otra muestra del contador en `x1` para usarla como semilla, espera a que se liberen los switches, pone en cero `x7` y `x12`,

muestra `00` e inicia otra ronda. Este reinicio se realiza mediante Assembly; no activa el reset físico del procesador ni reinicia el contador libre.

3.10. Rebotes y pulsaciones anticipadas

El Assembly no implementa un filtro de rebote temporal para los switches. Los bucles de liberación ayudan a separar etapas, pero no filtran rebotes eléctricos.

Durante la espera de tres segundos no se consultan los switches. Al entrar en `wait_for_answer`, se acepta la primera lectura no nula, incluso si el jugador ya mantenía presionado un switch antes de encenderse el LED objetivo. El programa no exige explícitamente detectar una transición nueva después de encender el objetivo.

4. Temporización y cálculos

4.1. Frecuencia y constantes

El juego utiliza como referencia una frecuencia de incremento del contador de 25 MHz. El período de un ciclo es:

$$T_C = \frac{1}{25\,000\,000}\ \text{s} = 40\ \text{ns}.$$

El número de ciclos para medir un intervalo t se calcula como $N = tf_C$. De ahí se obtienen las constantes usadas por el programa:

Tabla 3: Constantes temporales para un contador de 25 MHz.

Uso	**Intervalo**	**Ciclos**
Espera antes de cada objetivo	3 s	75 000 000
Unidad para cuantizar respuestas	0,1 s	2 500 000
Pausa antes del promedio	2 s	50 000 000

El display expresa décimas de segundo: `25` representa 2,5 segundos, no 25 segundos. La unidad se mantiene para los resultados individuales y para el promedio.

4.2. Resta modular y desbordamiento

Si C_i es la muestra inicial y C_f la muestra final, la diferencia de 32 bits es:

$$\Delta C = (C_f - C_i) \text{ mód } 2^{32}.$$

El contador vuelve a cero al superar $2^{32}-1$. La resta modular permite medir correctamente un intervalo que cruce ese punto, siempre que el tiempo transcurrido sea menor que un período completo del contador.

Por ejemplo, con $C_i =$ `0xFFFFFFF0` y $C_f =$ `0x00000020`, la resta de 32 bits produce `0x30` = 48 ciclos. A 25 MHz, el período completo del contador es:

$$\frac{2^{32}}{25\,000\,000} = 171{,}79869184\,\text{s}.$$

Las esperas de tres y dos segundos son menores que ese período. Si una respuesta tarda un período completo o más, las dos muestras por sí solas ya no permiten reconstruir todo el intervalo. La instrucción `bltu` compara los valores como cantidades sin signo, de acuerdo con su uso como conteos de ciclos.

4.3. Selección pseudoaleatoria del LED

La semilla s es la muestra del contador guardada en `x1` al detectar la pulsación de inicio. En cada ronda, C_r es la muestra reciente del contador al terminar la espera y c es la cantidad de aciertos guardada en `x7`. El índice se calcula como:

$$j = (s + C_r + c) \text{ mód } 4, \qquad j \in \{0, 1, 2, 3\}.$$

La instrucción `andi x3, x3, 3` conserva los dos bits inferiores de la suma. Las ramas convierten el índice en una máscara de LED:

Tabla 4: Relación entre índice, máscara y LED objetivo.

Índice	**Máscara**	**Bits 3 a 0**
0	1	`0001`
1	2	`0010`
2	4	`0100`
3	8	`1000`

El método es determinista: los mismos valores s, C_r y c producen el mismo LED. Solo influyen sus dos bits inferiores en la selección. Por ello, el método puede variar entre partidas y rondas, pero no garantiza por sí solo una distribución uniforme ni evita todos los patrones.

5. Diseño e implementación del assembler

Se desarrolló un assembler en Rust para traducir programas escritos en lenguaje ensamblador RISC-V a código máquina compatible con la arquitectura RV32E utilizada por el procesador Espino Core. La implementación admite instrucciones de los formatos R, I, S, B, U y J, restringiendo el uso de registros al conjunto disponible en RV32E (x0–x15).

El ensamblado se realiza mediante un algoritmo de dos pasadas. En la primera se procesan las instrucciones, se identifican las etiquetas y se construye una tabla de símbolos con sus respectivas direcciones. En la segunda se resuelven las referencias a dichas etiquetas y se codifican las instrucciones mediante operaciones bit a bit sobre enteros de 32 bits, posicionando los campos correspondientes según el formato de cada instrucción.

Adicionalmente, se implementó la expansión de pseudoinstrucciones comunes, así como mecanismos de validación de registros, operandos e inmediatos, permitiendo detectar errores en el código fuente antes de generar el archivo de salida.

5.1. Flujo de compilación y programación

El assembler se integró al sistema de compilación mediante el archivo `Makefile` principal, reemplazando la implementación anterior desarrollada en Python.

El proceso comienza con el archivo `sw/game.s`, que contiene el programa en lenguaje ensamblador. Mediante Cargo, se ejecuta el assembler en Rust para generar `sw/game.hex`, donde cada línea representa una instrucción de máquina de 32 bits codificada en hexadecimal.

Posteriormente, el flujo continúa con la síntesis del diseño mediante Yosys, la ubicación y el enrutamiento mediante NextPNR y la generación del bitstream mediante Icepack. Finalmente, Iceprog permite programar la FPGA con el archivo generado.

La integración se verificó mediante la generación exitosa del archivo hexadecimal y del bitstream. La programación física de la FPGA quedó pendiente de validación, dado que la placa no se encontraba conectada durante las pruebas.

6. Validación y resultados en FPGA

La inspección del código permite describir el comportamiento esperado, pero no sustituye la comprobación en la placa. El material usado para completar este informe no incluía capturas ni un registro de pruebas ejecutadas en la FPGA. Por ese motivo, la Tabla 5 distingue los criterios de prueba del estado de su evidencia; no se presentan mediciones no documentadas como resultados obtenidos.

Tabla 5: Casos funcionales y evidencia necesaria para cerrar la validación.

Caso	Comportamiento esperado	Estado de evidencia
Inicio y espera	Tras iniciar y soltar el switch, los cuatro LEDs permanecen encendidos durante al menos tres segundos antes de seleccionarse un objetivo.	Requiere observación cronometrada en la FPGA.
Respuesta correcta	Se apaga el LED objetivo y el display muestra el tiempo en décimas, empaquetado como dos cifras.	Requiere probar los switches y registrar la salida del display.
Respuesta incorrecta	Se apagan los LEDs, se muestra `EE` y la ronda vuelve a comenzar sin borrar los aciertos anteriores.	Requiere provocar un error y observar que el conteo se conserva.
Décima respuesta	Tras liberar el switch, el último resultado permanece visible durante la pausa programada de dos segundos.	Requiere medición en la FPGA.
Promedio	El display presenta la suma de diez resultados dividida entre diez, en décimas y con truncamiento entero.	Requiere comparar con diez tiempos registrados.
Nueva partida	Una pulsación después del promedio toma una nueva semilla, borra los acumuladores y reinicia el juego.	Requiere observar una segunda partida.

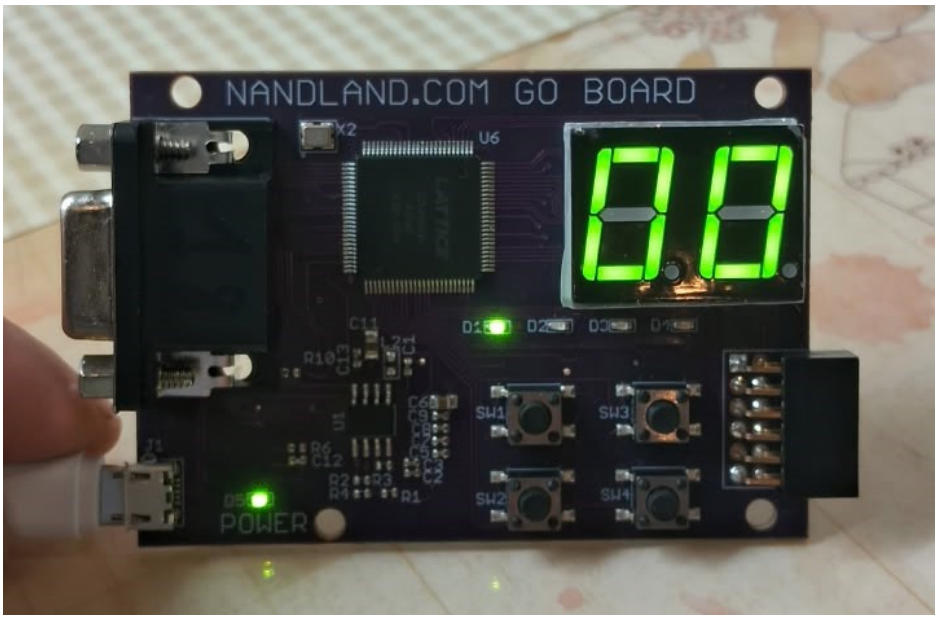


Figura 1: En medio de una ronda

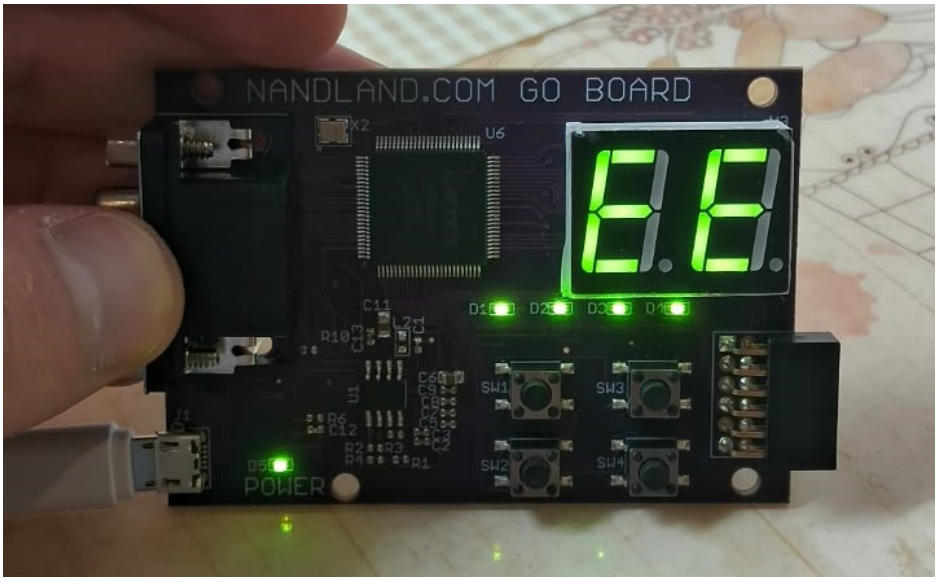


Figura 2: Error

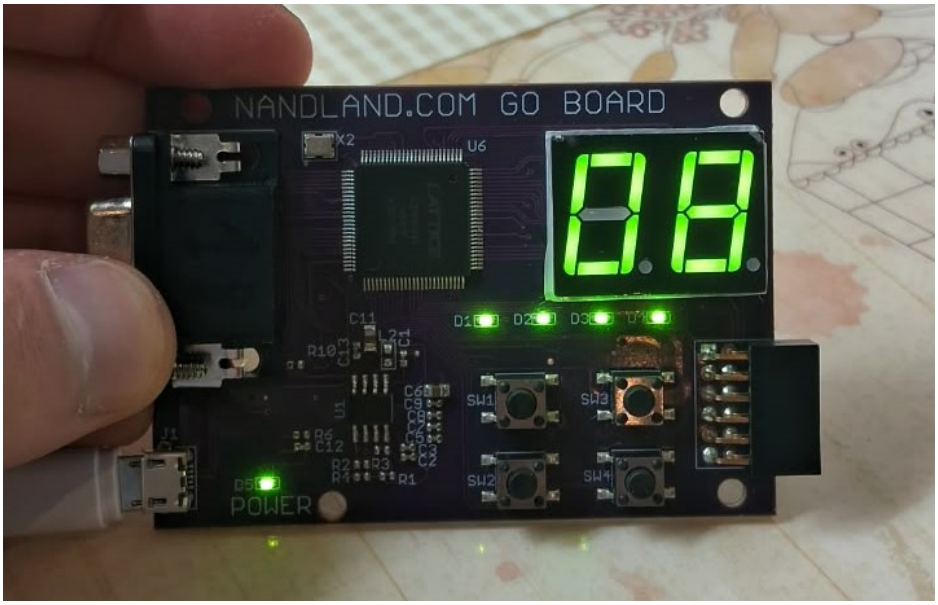


Figura 3: Promedio

6.1. Limitaciones observables

La implementación no incluye un filtro temporal de rebote para los switches. Además, acepta la primera lectura no nula: si un switch permanece presionado desde antes de encenderse el LED objetivo, puede contarse como respuesta sin una nueva transición. El selector del LED es determinista y usa solo dos bits de la suma de muestras, por lo que no garantiza una distribución uniforme. El tiempo medido incluye la latencia del sondeo en software. El display limita cada resultado a 99 décimas, pero la espera por una respuesta no tiene un timeout.

7. Conclusiones

El proyecto integra un juego de reacción mediante una separación de responsabilidades: el hardware proporciona memoria, MMIO, entradas y un contador libre; el programa Assembly administra las rondas, mide los intervalos y calcula el promedio. La solución utiliza los registros `x0` a `x15`, accesos alineados de 32 bits y las instrucciones codificadas por el assembler y ejecutables por el núcleo.

Las constantes temporales son coherentes con un contador de 25 MHz: tres segundos para la espera inicial, una décima para cuantizar el tiempo y dos segundos antes del promedio. Una respuesta incorrecta repite el intento sin alterar los resultados acumulados. Diez respuestas correctas habilitan el promedio, que se presenta en décimas de segundo.

Para cerrar la validación experimental se deben ejecutar los casos de la Tabla 5 en la GoBoard y agregar capturas del inicio, una respuesta correcta, un error y el promedio. También se debe registrar una serie de diez tiempos para comprobar el cálculo del promedio.

Referencias

[1] Universidad de los Andes, Facultad de Ingeniería y Ciencias Aplicadas, *Proyecto 2: Enunciado*, Arquitectura de Computadores, semestre 2026-2.

[2] Repositorio del proyecto, *Pochoco SoC & Espino Core*, archivo `README.md`.

[3] Repositorio del proyecto, archivo `rtl/pochoco_soc.v`.

[4] Repositorio del proyecto, archivo `rtl/game_top.v`.

[5] Repositorio del proyecto, archivos `sw/assembler.py` y `sw/game_1.s`.

[6] Repositorio del proyecto, archivo `Makefile`.